

Spring Boot CRUD Project — Database, Entity, Repository, Service, and Controller

Coder Army

Why Are We Not Creating the Table Manually?

Normally, when working directly with SQL, we manually create tables using queries such as:

```
CREATE TABLE students (...);
```

But in this project, we are using **JPA with Hibernate**.

JPA allows us to work with Java classes and objects, while Hibernate can map those classes to database tables.

So instead of manually creating the `students` table, Hibernate can look at our entity class and create or update the table structure based on our configuration.

```
spring.jpa.hibernate.ddl-auto=update
```

This means Hibernate will try to update the database schema based on the entity class.

For example:

```
@Entity
@Table(name = "students")
public class Student {
    // fields
}
```

Hibernate can use this class to create or update a `students` table.

Important Point

Hibernate can create or update tables, but the database itself should already exist.

So we still need to create the database once:

```
CREATE DATABASE student_crud_db;
```

After that, Hibernate can create/update tables inside this database.

Basic MySQL Configuration

Inside `application.properties`, add the following configuration:

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_crud_db
spring.datasource.username=root
spring.datasource.password=your_password

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Understanding Each Property

1. `spring.datasource.url`

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_crud_db
```

This tells Spring Boot where the database is located.

URL Breakdown

Part	Meaning
<code>jdbc</code>	Java Database Connectivity URL format
<code>mysql</code>	Database type is MySQL
<code>localhost</code>	MySQL is running on the same system
<code>3306</code>	Default MySQL port
<code>student_crud_db</code>	Database name

So this URL means:

Connect this Spring Boot application to the MySQL database named `student_crud_db` running on the local machine at port `3306`.

2. `spring.datasource.username`

```
spring.datasource.username=root
```

This is the MySQL username.

In local development, we commonly use the `root` user.

3. `spring.datasource.password`

```
spring.datasource.password=your_password
```

This is the password of the MySQL user.

For example, if the MySQL password is `123456`, then write:

```
spring.datasource.password=123456
```

4. `spring.jpa.hibernate.ddl-auto`

```
spring.jpa.hibernate.ddl-auto=update
```

This property controls how Hibernate handles database table structure.

With `update`, Hibernate checks the entity class and tries to update the table structure accordingly.

For example:

- If the table does not exist, Hibernate can create it.
- If a new field is added in the entity class, Hibernate can try to add a new column.
- Existing data is usually not deleted with `update`.

Other `ddl-auto` Values

Value	Meaning
<code>none</code>	Hibernate will not manage the table structure
<code>validate</code>	Hibernate only checks whether the table matches the entity
<code>update</code>	Hibernate updates the table structure based on the entity
<code>create</code>	Drops old tables and creates fresh tables every time the app starts
<code>create-drop</code>	Creates tables when the app starts and drops them when the app stops

Production Note

In real production projects, database changes are usually not handled using:

```
spring.jpa.hibernate.ddl-auto=update
```

Instead, teams use controlled migration tools such as:

```
Flyway  
Liquibase
```

SQL migration scripts

This gives better control over database changes.

For learning and local development, `ddl-auto=update` is useful and convenient.

5. `spring.jpa.show-sql`

```
spring.jpa.show-sql=true
```

This tells Hibernate to print SQL queries in the console.

For example, when we save a student, Hibernate may print an `insert` query.

This helps us understand what SQL Hibernate is generating internally.

6. `spring.jpa.properties.hibernate.format_sql`

```
spring.jpa.properties.hibernate.format_sql=true
```

This formats SQL queries properly in the console.

Without formatting, the SQL query may appear in one long line.

With formatting, it becomes more readable.

7. Driver Class Configuration

In most Spring Boot projects, we do not need to manually write the driver class.

Spring Boot can detect the driver from:

1. MySQL dependency
2. JDBC URL

But if someone faces driver-related issues, they can add:

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Java Objects and Database Tables

In Java, we work with classes and objects.

In SQL databases, we work with tables and rows.

This creates a natural mismatch:

```
Java world      → classes and objects
Database world  → tables and rows
```

Now the question is:

| How will a Java object become a database row?

This is where JPA helps.

JPA provides **Object-Relational Mapping**, commonly called ORM.

ORM means mapping Java classes to relational database tables.

Example:

```
Student class   → students table
Student object  → one row inside students table
Student fields  → columns of the table
```

So when we create:

```
Student student = new Student();
```

we are creating a Java object.

When JPA saves this object, it can become a row inside the `students` table.

Entity Class

An entity is a Java class that represents a database table.

Example:

```
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private Integer age;
    private String mobile;
    private Boolean isDeleted = false;
}
```

Important Entity Annotations

1. @Entity

@Entity

@Entity tells JPA that this class should be treated as a database entity.

Without @Entity, JPA treats the class as a normal Java class.

With @Entity, JPA understands:

This class can be mapped to a database table.
Objects of this class can be saved as database rows.

2. @Table

```
@Table(name = "students")
```

`@Table` tells JPA the name of the database table.

If we do not use `@Table`, JPA usually uses the class name as the table name.

Example:

```
public class Student
```

may be mapped to a table named:

```
student
```

By using `@Table`, we can control the table name explicitly.

3. @Id

```
@Id  
private Long id;
```

Every table should have a primary key.

A primary key uniquely identifies each row.

In our `Student` entity, `id` is the primary key.

4. @GeneratedValue

```
@GeneratedValue(strategy = GenerationType.IDENTITY)  
private Long id;
```

We do not want to manually provide the ID every time we create a student.

`@GeneratedValue` tells JPA that the ID should be generated automatically.

`GenerationType.IDENTITY` means the database will handle ID generation.

In MySQL, this usually means an auto-increment column.

What Is Repository?

Our Spring Boot application works with Java objects.

Example:

```
Student student = new Student();
student.setName("Rahul Sharma");
student.setEmail("rahul@example.com");
student.setAge(22);
```

But the actual data should be stored in MySQL.

So we need a layer that can perform database operations such as:

```
Save student into database
Fetch students from database
Find student by ID
Update student
Delete student
```

This responsibility belongs to the **Repository layer**.

Creating Repository

```
public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

Here, we are creating an interface, not a class.

Normally, when we create an interface, we need to manually create its implementation class.

But with Spring Data JPA, we do not manually create the implementation.

Spring Data JPA creates the implementation automatically at runtime.

Understanding `JpaRepository<Student, Long>`

`JpaRepository` needs two things:

1. Entity type
2. Primary key type

In our case:

Student → Entity class
Long → Type of primary key/id

So this means:

`JpaRepository<Student, Long>`

will provide database operations for the `Student` entity, whose ID type is `Long`.

Do We Need `@Repository` ?

For Spring Data JPA repositories, we usually do not need to write:

`@Repository`

Spring Data JPA automatically detects interfaces that extend `JpaRepository`.

So this is enough:

```
public interface StudentRepository extends JpaRepository<Student, Long> {  
    }  
}
```

Important Repository Methods

`JpaRepository` gives us many ready-made methods.

For this CRUD project, these methods are important:

```
save()  
findAll()  
findById()  
deleteById()  
existsById()
```

1. `save()`

```
studentRepository.save(student);
```

`save()` is used to save data in the database.

It can do two things:

```
If the student is new      → insert new row  
If the student already exists → update existing row
```

Example:

```
public Student createStudent(Student student) {  
    return studentRepository.save(student);  
}
```

If the student does not have an ID, JPA treats it as a new record and inserts it.

2. `findAll()`

```
studentRepository.findAll();
```

This fetches all rows from the `students` table.

But in our project, we are using soft delete.

So if we directly use:

```
studentRepository.findAll();
```

it may return both:

```
Active students  
Soft-deleted students
```

Usually, when the user calls:

```
GET /api/students
```

we want only active students.

So instead of `findAll()`, we can create a custom method:

```
List<Student> findByIsDeletedFalse();
```

Then we can fetch only those students whose `isDeleted` value is `false`.

Example:

```
public interface StudentRepository extends JpaRepository<Student, Long> {  
    List<Student> findByIsDeletedFalse();  
}
```

3. `findById()`

```
studentRepository.findById(id);
```

This fetches one student by ID.

Return type:

```
Optional<Student>
```

Example:

```
Optional<Student> student = studentRepository.findById(1L);
```

Why does it return `Optional`?

Because the student may or may not exist.

If the student exists, we get the student object.

If the student does not exist, the `Optional` is empty.

So `findById()` forces us to handle this case properly:

```
What if the data is not found?
```

This helps us write safer code.

4. `deleteById()`

```
studentRepository.deleteById(id);
```

This performs hard delete.

It removes the row permanently from the database.

Example:

```
studentRepository.deleteById(1L);
```

After this, student ID 1 is removed from the table.

This can be used for an API like:

```
DELETE /api/students/{id}
```

5. `existsById()`

```
studentRepository.existsById(id);
```

This checks whether a student exists with the given ID.

It returns a boolean value:

```
boolean exists = studentRepository.existsById(1L);
```

This is useful before hard delete.

Example:

```
public boolean deleteStudent(Long id) {  
    if (!studentRepository.existsById(id)) {  
        return false;  
    }  
  
    studentRepository.deleteById(id);  
    return true;  
}
```

This avoids trying to delete a record that does not exist.

Service Layer

The service layer contains the business logic of the application.

The controller should not directly contain all business logic.

The controller should mainly handle HTTP requests and responses.

The service layer should handle application logic such as:

```
Create student
Update student
Fetch active students
Soft delete student
Hard delete student
Check whether student exists
```

@Service

```
@Service
public class StudentService {
}
```

`@Service` tells Spring that this class belongs to the service layer and Spring should create its object as a bean.

`@Service` is similar to `@Component`, but it is more specific.

```
@Component → generic Spring-managed class
@Service    → Spring-managed class that contains business logic
```

Using `@Service` makes the code more readable and clearly shows the responsibility of the class.

What Is Controller?

The repository talks to the database.

The service contains business logic.

But an external client, like Postman, browser, frontend application, or mobile app, cannot directly call the repository or service.

So we need a controller.

The controller is the layer that receives HTTP requests and sends HTTP responses.

Example:

```
GET /api/students
POST /api/students
PUT /api/students/{id}
DELETE /api/students/{id}
```

These requests are handled by the controller.

@RestController

```
@RestController
public class StudentController {
}
```

`@RestController` tells Spring that this class will handle REST API requests.

It is used when we want to return data directly as the response, usually in JSON format.

For example, if our method returns a `Student` object:

```
return student;
```

Spring Boot can automatically convert it into JSON:

```
{
  "id": 1,
  "name": "Rahul Sharma",
  "email": "rahul@example.com",
}
```



```
"age": 22,  
"mobile": "9876543210",  
"isDeleted": false  
}
```

This conversion is usually done using Jackson, which comes automatically with Spring Boot web dependency.

@RequestMapping

```
@RequestMapping("/api/students")
```

`@RequestMapping` defines the base URL for all APIs inside the controller.

Example:

```
@RestController  
@RequestMapping("/api/students")  
public class StudentController {  
}
```

Now all APIs inside this controller will start with:

```
/api/students
```

So instead of writing `/api/students` again and again on every method, we define it once at the class level.

Why Use ResponseEntity ?

In simple tutorials, we may see code like this:

```
public Student getStudentById(@PathVariable Long id) {  
    return studentService.getStudentById(id).get();  
}
```

```
}
```

This works only when the student exists.

But what if the student does not exist?

Then our API should return a proper HTTP status code.

Example:

```
Student found      → 200 OK  
Student not found → 404 Not Found
```

`ResponseEntity` helps us control:

```
Response body  
HTTP status code  
Headers, if needed
```

Example: Returning 200 OK

```
return ResponseEntity.ok(student);
```

Meaning:

```
Status: 200 OK  
Body: student data
```

Example: Returning 404 Not Found

```
return ResponseEntity.notFound().build();
```

Meaning:

Status: 404 Not Found
Body: empty

Example: Returning 201 Created

When a new student is created, we can return `201 Created`.

```
return ResponseEntity.status(HttpStatus.CREATED).body(created  
Student);
```

Meaning:

Status: 201 Created
Body: created student data

This is more meaningful than always returning `200 OK`.

Complete Flow of Request

When a client sends a request:

POST /api/students

the flow is:

```
Client/Postman  
  ↓  
Controller  
  ↓  
Service  
  ↓  
Repository  
  ↓  
Database
```

After saving the data, the response goes back:

```
Database
  ↓
Repository
  ↓
Service
  ↓
Controller
  ↓
Client/Postman
```

Layer Responsibilities

Layer	Responsibility
Controller	Handles HTTP requests and responses
Service	Contains business logic
Repository	Talks to the database
Entity	Represents database table structure
Database	Stores actual data

Summary

In this CRUD project:

- MySQL stores the actual student data.
- `application.properties` connects Spring Boot with MySQL.
- JPA maps Java classes to database tables.
- Hibernate can create/update tables using `ddl-auto=update`.
- `@Entity` marks a class as a database entity.
- `@Table` defines the table name.
- `@Id` marks the primary key.

- `@GeneratedValue` allows automatic ID generation.
- Repository layer performs database operations.
- Service layer contains business logic.
- Controller layer handles API requests and responses.
- `ResponseEntity` helps return proper HTTP status codes.

The main idea is:

Java object gets converted into database row using JPA.

And the complete Spring Boot CRUD flow becomes:

Controller → Service → Repository → Database